CHAPITRE 2:

Le langage de programmation JAVA

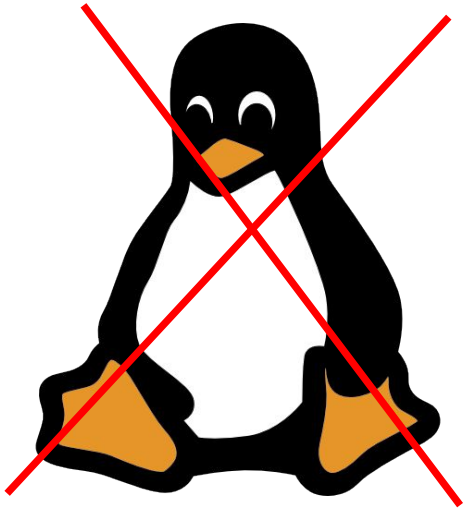
Justin Sullivan / Getty Images

199
1

Langage
Portable

The Green Project

1996



Java

Mac OS

Microsoft Windows

CARACTÉRISTIQUES DE JAVA

- **Java est portable**

La grande force de Java réside dans sa capacité à permettre aux développeurs d'écrire un programme une seule fois, puis de l'exécuter sur n'importe quelle plateforme sans avoir besoin de le modifier pour chaque système spécifique.

JAVA EST PORTABLE

1. **Compilation en bytecode** : Ce bytecode est un ensemble d'instructions standardisées pour la Java Virtual Machine (JVM), pas pour une machine spécifique.

2. **Exécution par la JVM** : Pour exécuter du bytecode, une plateforme doit avoir une JVM. Cela signifie que le même fichier bytecode Java peut être exécuté sur n'importe quelle plateforme qui dispose d'une JVM compatible, sans modification.

JAVA EST PORTABLE

Exemple :

- C'est comme si vous aviez une clé universelle qui ouvre de nombreuses serrures différentes.

- Au lieu d'avoir besoin d'une clé différente pour chaque serrure.

- Cela rend Java très flexible et pratique pour développer des applications qui peuvent fonctionner sur différents appareils et systèmes d'exploitation.

JAVA EST PORTABLE (EXEMPLE 2)

Jeu vidéo:

1. **Développement initial** : Vous développez ce jeu sur votre ordinateur personnel qui fonctionne sous Windows. Vous utilisez l'environnement de développement intégré (IDE) de Java pour écrire, compiler et tester votre jeu. Une fois satisfait, vous générez le fichier exécutable (sous forme de fichiers **.jar** ou **.class** contenant le bytecode Java).

2. **Portabilité** : Vous décidez maintenant de partager ce jeu avec vos amis qui utilisent différents systèmes d'exploitation : l'un a un Mac, l'autre utilise Linux, et un troisième a même un système d'exploitation moins courant comme Solaris.

JAVA EST PORTABLE (SUITE EXEMPLE 2)

3. **Exécution sans modification** : Chacun de vos amis télécharge votre jeu. Malgré la diversité de leurs systèmes d'exploitation, ils n'ont pas besoin de versions spécifiques du jeu pour leur propre système. Tout ce dont ils ont besoin est une machine avec une JVM compatible. Ils lancent la JVM, ouvrent votre fichier de jeu, et le bytecode Java est interprété (ou compilé à la volée via JIT) en instructions spécifiques à leur système d'exploitation et matériel.

4. **Expérience uniforme** : Tous vos amis peuvent jouer au jeu exactement de la même manière que vous sur votre machine Windows, sans perte de fonctionnalités ou de performance. Les graphiques, les contrôles et la logique du jeu restent cohérents sur toutes les plateformes.

CARACTÉRISTIQUES DE JAVA

Java est orienté Objet

- Comme la plupart des langages récents, Java est orienté objet.
- Chaque fichier source contient la définition d'une ou plusieurs classes qui sont utilisées les unes avec les autres

pour former une application.

- Java n'est pas complètement objet car il définit des types primitifs (entier, caractère, flottant, booléen,...).

JAVA EST ORIENTÉ OBJET (EXEMPLE)

- Java est comme une grande usine où chaque machine (classe) a un rôle spécifique à jouer dans la production d'un produit (l'application).
- Ces machines (classes) peuvent créer des pièces (objets) qui sont utilisées et réutilisées dans différents processus de l'usine pour accomplir diverses tâches.
- Bien que l'usine soit moderne et utilise principalement des machines pour tout faire (approche orientée objet), elle utilise encore quelques outils manuels simples (types primitifs) pour certaines tâches spécifiques, car ils sont rapides et efficaces pour de petits travaux.

CARACTÉRISTIQUES DE JAVA

- **JAVA est simple**
  - Java a été conçu pour être un langage simple et facile à utiliser, en éliminant certains concepts complexes présents dans d'autres langages de programmation.
  - Les créateurs de Java ont décidé de ne pas inclure certaines fonctionnalités qui pourraient rendre la programmation plus compliquée ou source d'erreurs.

JAVA EST SIMPLE

1. Absence de Pointeurs

- **Pointeurs dans d'autres langages** :

  puissant mais aussi risqué.

- **Approche de Java** :

  - Références aux objets,

  - Plus sûr parce que le langage et l'environnement d'exécution (JVM) gèrent automatiquement la localisation et la gestion de la mémoire.

  - Réduit les risques de problèmes de sécurité comme les dépassements de tampon ou les fuites de mémoire.

JAVA EST SIMPLE

2. Héritage Multiple

- **Héritage Multiple** :
  - L'héritage multiple permet à une classe d'hériter des propriétés et méthodes de plus d'une classe parente.
  - Bien que puissant, cela peut conduire à des complications, notamment le problème du diamant.
- **Approche de Java** :
  - Java permet l'héritage simple, où une classe ne peut hériter que d'une seule classe parente.
  - Pour gérer les situations où l'héritage multiple pourrait être utile, Java utilise les interfaces, qui permettent à une classe d'implémenter plusieurs ensembles de méthodes sans hériter de l'implémentation de ces méthodes.

CARACTÉRISTIQUES DE JAVA

- **JAVA est fortement typé**
  - Java est un langage fortement typé, ce qui signifie que chaque variable a un type défini explicitement et ce type est vérifié au moment de la compilation.
  - Cette caractéristique a des implications importantes pour la conversion entre différents types de données

JAVA EST FORTEMENT TYPÉ

Typage Fort:

- **Définition explicite** : Lors de la déclaration d'une variable en Java, vous devez spécifier son type, qu'il s'agisse d'un type primitif (comme **int**, **double**, **char**, etc.) ou d'un type objet (comme **String**, **Array**, **List**, etc.).

- **Vérification au moment de la compilation** : Le compilateur Java vérifie que toutes les opérations sur les variables sont compatibles avec leurs types, ce qui aide à prévenir les erreurs d'exécution.

Pas de Conversion Automatique Risquée:

- **Conversions sûres** : Java autorise certaines conversions automatiques entre types de données quand elles ne risquent pas de perte d'informations, comme la conversion d'un **int** en **long** ou d'un **float** en **double**.

- **Conversions à risque** : Pour les conversions qui pourraient entraîner une perte de données, comme la conversion d'un **double** en **int** ou d'un **long** en **short**, Java exige une action explicite de la part du développeur pour éviter des conversions accidentelles et potentiellement erronées.

JAVA EST FORTEMENT TYPÉ

Casts et Méthodes de Conversion

- **Casts explicites** : Lorsqu'une conversion à risque est nécessaire, vous devez utiliser un cast explicite pour indiquer au compilateur que vous êtes conscient de la conversion et que vous acceptez les risques potentiels. Par exemple, **(int) myDouble;** convertit explicitement une variable **double** en **int**, en tronquant les décimales.

- **Méthodes statiques** : Java fournit également des méthodes statiques dans des classes comme **Integer**, **Double**, etc., pour convertir entre différents types de manière plus contrôlée. Par exemple, **Integer.parseInt("123")** convertit une chaîne de caractères en un entier.

Environnement JAVA:

La JVM (**Java Virtual Machine)** est le moteur qui exécute le bytecode Java. Elle est responsable de la conversion du bytecode en instructions machine nativement exécutables par le matériel spécifique sur lequel elle s'exécute.

- La JVM rend les applications Java indépendantes de la plateforme, car le bytecode Java peut s'exécuter sur n'importe quelle JVM, indépendamment du système d'exploitation ou de l'architecture matérielle.

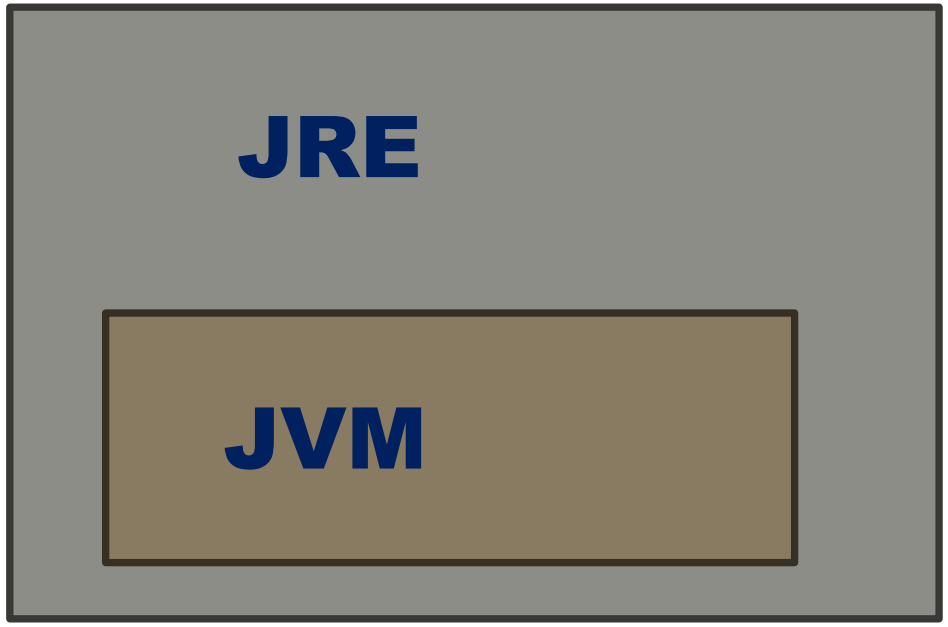
- La JVM gère également des aspects clés tels que la gestion de la mémoire (y compris le ramasse-miettes), la sécurité et l'optimisation de l'exécution du code via des techniques comme la compilation Just-In-Time (JIT).

Environnement JAVA:

Le JRE **(Java Runtime Environment)** est l'ensemble des logiciels nécessaires pour exécuter des applications Java. Il est spécifiquement conçu pour fournir les bibliothèques de classes Java, la JVM et d'autres composants nécessaires pour

- Le JRE permet aux applications Java de s'exécuter sur n'importe quel appareil ou système d'exploitation sans nécessiter de modifications, incarnant ainsi le principe "Write Once, Run Anywhere" (WORA) de Java.

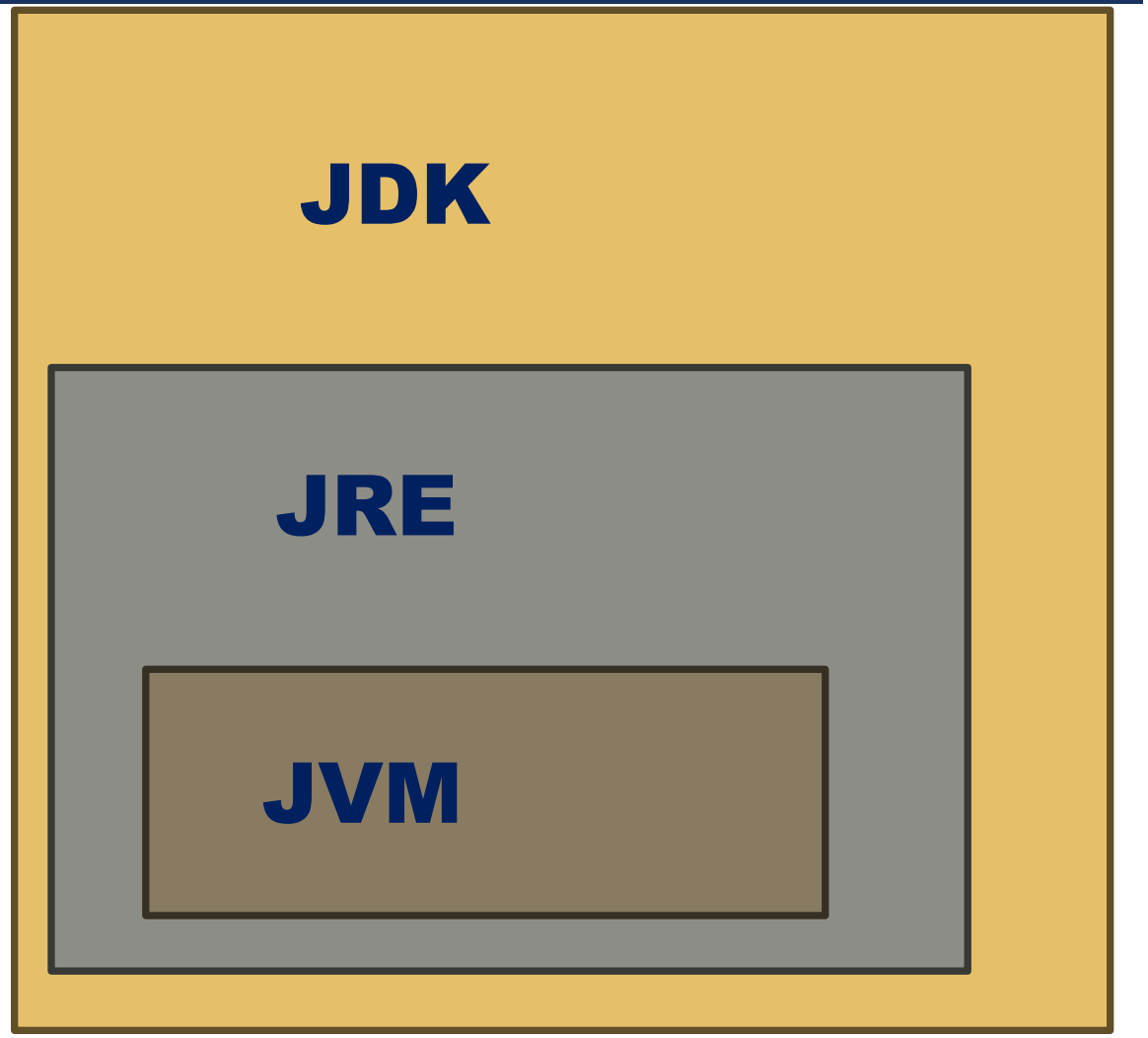
- Le JRE comprend la JVM, les bibliothèques de classes de l'API Java et d'autres fichiers nécessaires à l'exécution des applications Java.



Environnement JAVA:

Le JDK **(Java Development Kit)** est l'ensemble complet d'outils de développement pour Java. Il inclut tout ce dont vous avez besoin pour écrire, compiler et déboguer des applications Java.

Le JDK comprend le compilateur Java (**javac**), l'interpréteur Java (**java**), d'autres outils de développement (comme **javadoc** pour la documentation et **jdb** pour le débogage), ainsi que la bibliothèque standard Java (le JRE).

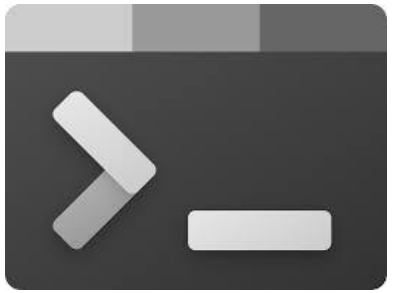


Environnement JAVA:

Afin d'écrire un programme en JAVA, il est possible de suivre deux procédures différentes :

- **implémenter le code dans un simple éditeur -> compiler et exécuter ce programme en ligne de commande**

Compiler et exécuter



- **Utiliser un environnement de développement intégré**

Etapes d'exécution d'une Application java :

1- une ou plusieurs classes .java

2- compilation grâce à javac

3-obtension de fichiers .class

4- interprétation du byte code par la machine virtuelle avec java

5- Obtention d'un langage machine compréhensible par le

Code source	MyProg.java
Compilateur	Javac MyProg.java
Byte code	MyProg.class
interpréteur	Java MyProg
Code machine	Virtualisation
Système d'exploitation	Système d'exploitation

Règles syntaxique de base pour la programmation JAVA

Sensibilité à la casse:

- Java est sensible à la casse, ce qui signifie que les majuscules et minuscules sont distinguées.
- Par exemple, **Voiture**, **voiture** et **VOITURE** sont considérées comme trois identifiants différents.

Règles syntaxique de base pour la programmation JAVA :

Structure de base d'une classe en JAVA :

```java
public class NomDeLaClasse {
    // Variables d'instance
    // Constructeurs
    // Méthodes
}
```

- Chaque fichier **.java** contient au moins une classe, et chaque classe contient des variables et des méthodes.
- Pour exécuter une application Java, la classe doit contenir une méthode main comme point d'entrée du programme. La signature de cette méthode doit être **public static void**

Règles syntaxique de base pour la programmation JAVA :

Les blocs de codes :

En Java, les blocs de code sont délimités par des accolades { } et sont utilisés pour définir le corps des classes, des méthodes et pour regrouper des instructions dans des structures de contrôle comme les boucles et les conditions.

```
public class BlocExemple {

    public static void main(String[] args) {
        // Bloc de la méthode main
        int x = 10;

        if (x == 10) {
            // Bloc conditionnel
            System.out.println("x est égal à 10");
        }
    }
}
```

Règles syntaxique de base pour la programmation JAVA:

Les commentaires:

- Un commentaire sur une ligne commence par // et continue jusqu'à la fin de la ligne :

```
// ceci est un commentaire
double i = 0.0; // ceci est également un commentaire
```

- Un commentaire sur plusieurs lignes commence par /* et se termine par */ :

```
/* ceci est un commentaire
   sur plusieurs lignes */
double i = 0.0;
```

Règles syntaxique de base pour la programmation JAVA :

Le formatage du code

Le compilateur Java n'impose pas de formatage particulier du code. Dans la mesure où une instruction se termine par un point-virgule et que les blocs sont délimités par des accolades, il est possible de présenter du code de façon différente.

```
double i = 0.0;
i = Math.sqrt(16);

if (i > 1) {
  i = -i;
}
```

```
double i=0.0;i=Math.sqrt(16);if(i>1){i=-i;}
```

Règles syntaxique de base pour la programmation JAVA :

Les conventions de nommage :

1. **Noms de classes et d'interfaces**
   - **Convention** : Les noms de classes et d'interfaces doivent commencer par une majuscule. Si le nom contient plusieurs mots, chaque mot doit également commencer par une majuscule (CamelCase).
   - **Exemples** : **Client**, **CompteBancaire**, **ListeObservable**.

2. **Noms de méthodes**
   - **Convention** : Les noms de méthodes doivent commencer par une lettre minuscule. Si le nom contient plusieurs mots, chaque mot suivant doit commencer par une majuscule (camelCase).
   - **Exemples** : **calculerTotal**, **lireDocument**, **envoyerMessage**.

Règles syntaxique de base pour la programmation JAVA :

3. Noms de variables

- **Convention** : Comme pour les méthodes, les noms de variables doivent également commencer par une lettre minuscule et suivre la notation camelCase pour les noms composés.
- **Déclaration de variables :** type nomVariable = valeur;
- **Exemples** : double **soldeCompte**, byte **age**, String **couleurFond**;

4. Constantes

- **Convention** : Les constantes doivent être écrites en majuscules. Si le nom contient plusieurs mots, ils doivent être séparés par des underscores (_).
- **Déclaration de constantes:** final type NOM_CONSTANTE = valeur;
- **Exemples** : final int MAX_USERS = 100; final boolean DEBUG_MODE = false;

Règles syntaxique de base pour la programmation JAVA :

Les mots-clés (mots réservés)

Comme pour la plupart des langages de programmation, il n'est pas possible d'utiliser comme nom dans un programme un mot-clé du langage.

La liste des mots-clés en Java est :

```
abstract    continue    for          new          switch
assert      default     if           package      synchronized
boolean     do          goto         private      this
break       double      implements   protected    throw
byte        else        import       public       throws
case        enum        instanceof   return       transient
catch       extends     int          short        try
char        final       interface    static       void
class       finally     long         strictfp     volatile
const       float       native       super        while
_ (underscore)
```

Le type **boolean**:

- En Java ce type est utilisé pour stocker des valeurs de vérité.
- Ce type est souvent utilisé dans les structures de contrôle pour tester des conditions et contrôler le flux d'exécution des programmes.

TYPES PRIMITIFS (BOOLEAN)

- **Valeurs possibles** : Une variable de type **boolean** peut uniquement prendre les valeurs **true** ou **false**.

- **Valeur par défaut** : Si un attribut de type **boolean** dans une classe n'est pas explicitement initialisé, il prend par défaut la valeur **false**.

- **Opérateurs applicables** : Les opérateurs de comparaison (**==**, **!=**) et l'opérateur logique de négation (**!**) peuvent être utilisés avec des variables de type **boolean**.
  NB:Les opérations arithmétiques ne sont pas applicables.

```java
public class BooleanExemple {
    public static void main(String[] args) {
        boolean estAllume = false;//Déclaration et initialisation d'une variable boolean

        if (!estAllume) { // Utilisation de l'opérateur de négation '!'
            System.out.println("La lumière est éteinte. On l'allume.");
            estAllume = true; // Modification de la valeur de la variable boolean
        }

        if (estAllume == true) {
            System.out.println("La lumière est maintenant allumée.");
        }

        if (estAllume) {
            System.out.println("Confirmation : la lumière est allumée.");
        }
    }
}
```

Le type **char**:

- **Stockage** : Chaque **char** est stocké sur 2 octets (16 bits), permettant de représenter des caractères en utilisant l'encodage UTF-16. Cela offre une plage de valeurs allant de 0 à 65,535 (2^16 - 1).

- **Valeur par défaut** : La valeur par défaut d'un attribut **char** non initialisé dans une classe est **\u0000**, qui correspond au caractère "null" (pas le même que la valeur **null** en Java).

- **Littéraux de caractères** : Les littéraux **char** sont spécifiés entre apostrophes, comme **‘a’, ‘c’…**

TYPES PRIMITIFS (CHAR)

Opérations arithmétiques sur char

- **Opérations autorisées** : Java permet les opérations arithmétiques sur les valeurs de type **char**. Ces opérations sont réalisées en utilisant la valeur numérique Unicode/UTF-16 du caractère.

```
char lettreA = 'a';
char lettreSuivante = (char) (lettreA + 1); // Incrémente pour obtenir 'b'

System.out.println("lettreA : " + lettreA); // Affiche : lettreA : a
System.out.println("lettreSuivante : " + lettreSuivante); // Affiche : lettreSuivante
```

TYPES PRIMITIFS (CHAR)

- **Parcours de l'alphabet** : il est possible de parcourir l'alphabet en incrémentant un **char** dans une boucle **for**.

```
for (char i = 'a'; i <= 'z'; ++i) {
    System.out.print(i + " "); // Affiche l'alphabet de a à z
}
```

TYPES PRIMITIFS (CHAR)

Affectation de nombres à char

- **Directe avec des littéraux** : Il est possible d'affecter directement un littéral numérique à une variable **char** si le littéral est dans la plage de valeurs valides pour **char**.

```
char a = 97; // 97 est le code UTF-16 pour 'a'
System.out.println("a : " + a); // Affiche : a : a
```

```
int codeA = 97;
char lettreA = (char) codeA; // Cast nécessaire car codeA est de type int
System.out.println("lettreA : " + lettreA); // Affiche : lettreA : a
```

pouvant y avoir une perte de précision ou une valeur hors plage.

- `byte` : 8 bits, range de -128 à 127.

```java
byte b = 100; // Exemple d'affectation pour byte
```

- `short` : 16 bits, range de -32,768 à 32,767.

```java
short s = 1000; // Exemple d'affectation pour short
```

- `int` : 32 bits, range de -2^31 à 2^31-1.

```java
int i = 100000; // Exemple d'affectation pour int
```

- `long` : 64 bits, range de -2^63 à 2^63-1.

```java
long l = 100000L; // Exemple d'affectation pour long
```

TYPES PRIMITIFS (ENTIERS)

- **Conversion Implicite**

  Java autorise la conversion implicite de types entiers lorsque l'on passe d'un type de plus petite taille à un type de plus grande taille

- **Transtypage Nécessaire**

  Lors de la conversion de types entiers d'une taille plus grande vers une taille plus petite, un transtypage explicite est nécessaire.

- **Affectation de Valeurs Littérales**

  Les valeurs littérales doivent être dans la plage de valeurs acceptables pour le type

TYPES PRIMITIFS

- **Types à Virgule Flottante:**
  - La catégorie réel permet l'emploi de nombres à virgule, appelés nombres réels ou encore flottants.
  - Deux types composent cette catégorie,
    - Le type float
    - Le type double

TYPES PRIMITIFS(VIRGULE FLOTTANTE)

- Une expression numérique de cette catégorie peut s'écrire en notation **décimale ou exponentielle**.
  - La notation décimale contient obligatoirement un point symbolisant le caractère « virgule » du chiffre à virgule.

    Exemple: Les valeurs 67.3 , -3. ou .64 sont des valeurs réelles utilisant la notation décimale.
  - La notation exponentielle utilise la lettre E pour déterminer où se trouve la valeur de l'exposant (puissance de 10).

    Exemple: Les valeurs 8.76E4 et 6.5E-12 sont des valeurs utilisant la notation exponentielle.

TYPES PRIMITIFS(VIRGULE FLOTTANTE)

- Une expression numérique de cette catégorie peut s'écrire en notation **décimale ou exponentielle**.
  - La notation décimale contient obligatoirement un point symbolisant le caractère « virgule » du chiffre à virgule.

    Exemple: Les valeurs 67.3 , -3. ou .64 sont des valeurs réelles utilisant la notation décimale.
  - La notation exponentielle utilise la lettre E pour déterminer où se trouve la valeur de l'exposant (puissance de 10).

    Exemple: Les valeurs 8.76E4 et 6.5E-12 sont des valeurs utilisant la notation exponentielle.

TYPES PRIMITIFS(VIRGULE FLOTTANTE)

- Dans les deux cas, le nombre réel est suivi de la lettre F (pour float) ou D (pour double).
- Les caractères minuscules f ou d sont également autorisés.
- La distinction entre float et double s'effectue sur le nombre d'octets utilisés pour coder l'information. Il en résulte une précision plus ou moins grande suivant le type employé

LES TYPES ÉLÉMENTAIRES ET LE TRANSTYPAGE:

type élémentaire	intervalle de variation	nombre de bits
boolean	false, true	1 bit
byte	[-128, +127]	8 bits
char	Caractères Unicode [0, 65536]	16 bits
double	Virgule flottante double précision $\sim 5.10^{308}$	64 bits
float	Virgule flottante simple précision $\sim 9.10^{18}$	32 bits
int	$[-2^{31}, 2^{31}-1]$	32 bits
long	$[-2^{63}, 2^{63}-1]$	64 bits
short	$[-2^{15}, 2^{15}-1]$	16 bits

COMMENT CHOISIR UN TYPE DE VARIABLE PLUTÔT QU'UN AUTRE ?

Sachant qu'une variable de type int (codée sur 4 octets) peut prendre toutes les valeurs de l'intervalle

(-2147483648, 214748364) et donc prendre, eu particulier, toutes les valeurs comprises entre -32768 et 32767 (type short) ou même entre -128 et 127 (type byte):

- Pourquoi ne pas déclarer toutes les variables entières d'un programme en type long (le type long nous offrant le plus grand choix de valeurs entières) ?
- Pourquoi ne pas déclarer les variables réelles d'un programme en type double plutôt qu'en float ?

LES TYPES RÉFÉRENCE

- En Java, en plus des types primitifs, il existe de nombreux types non primitifs (ou types référence) qui sont fréquemment utilisés.
- Ces types incluent les classes, les interfaces, les tableaux et les énumérations.

La classe « String »:

- La classe **String** en Java est utilisée pour stocker et manipuler des séquences de caractères. Elle est définie dans le package **java.lang**, qui est importé par défaut dans tous les programmes Java.
- Les objets **String** sont immuables, ce qui signifie que, une fois créés, leur contenu ne peut pas être modifié.
- Si vous effectuez une opération qui semble modifier une **String**, comme la concaténation, ce que Java fait réellement, c'est créer une nouvelle **String** avec le résultat de l'opération.

LES TYPES RÉFÉRENCE (STRING)

En Java, les chaînes de caractères (String) peuvent être créées de deux manières principales :

1. **Avec le mot-clé new :** Quand tu utilises **new String("Hello")**, Java crée un nouvel objet String dans la mémoire. Chaque fois que tu utilises **new**, un nouvel objet est créé, même si le contenu est identique à un objet String existant.
2. **Avec la syntaxe littérale :** Quand tu écris simplement **String s = "Hello";**, Java cherche d'abord dans un espace spécial de la mémoire appelé "pool de chaînes de caractères". Si la chaîne "Hello" existe déjà dans ce pool, Java réutilise cet objet String au lieu d'en créer un nouveau. Cela permet d'économiser de la mémoire, car les chaînes identiques partagent le même espace mémoire

LES TYPES RÉFÉRENCE (STRING)

La classe **String** fournit de nombreuses méthodes pour examiner et manipuler les séquences de caractères. Voici quelques méthodes courantes :

- **length()**: Retourne la longueur de la chaîne de caractères.
- **charAt(int index)**: Retourne le caractère à l'indice spécifié.
- **substring(int beginIndex, int endIndex)**: Retourne une nouvelle **String** qui est une sous-chaîne de cette **String**.

```java
public class ExempleChaine
{
    public static void main(String[] args) {
        String chaine = "Bonjour, Monde !";
        System.out.println("Longueur de la chaîne : " +chaine.length());
        System.out.println("Caractère à la position 5 : " + chaine.charAt(5));
        System.out.println("Sous-chaîne de l'indice 7 à 12 : " + chaine.substring(7, 12));
        System.out.println("La chaîne contient-elle 'Monde' ? " + chaine.contains("Monde"));
    }
}
```

LES TYPES RÉFÉRENCE (STRING)

- **Autres méthodes**
  - **equals(Object another)**: Compare cette **String** à l'objet spécifié.
  - **equalsIgnoreCase(String anotherString)**: Compare cette **String** à une autre **String**, en ignorant les différences de casse.
  - **startsWith(String prefix)**: Teste si cette **String** commence par le préfixe spécifié.
  - **endsWith(String suffix)**: Teste si cette **String** se termine par le suffixe spécifié.
  - **replace(char oldChar, char newChar)**: Retourne une nouvelle **String** résultant du remplacement de toutes les occurrences de **oldChar** dans cette **String** par **newChar**.
  - **trim()**: Retourne une copie de la **String**, avec tout espace blanc de début et de fin supprimé.
  - **split(String regex)**: Divise cette **String** autour des correspondances de l'expression régulière donnée

LES TYPES RÉFÉRENCE (STRING)

Concaténation

- En Java, la concaténation de **String** peut être effectuée en utilisant l'opérateur **+**. Par exemple, **"Hello " + "world!"** donnera **"Hello world!"**.
- Pour des concaténations multiples ou complexes, il peut être plus efficace d'utiliser un **StringBuilder** ou un **StringBuffer**.

Conversion

- Pour convertir des types primitifs ou des objets en **String**, vous pouvez utiliser la méthode **String.valueOf()**. Par exemple, **String.valueOf(123)** convertit l'entier **123** en **String**.

LES TYPES RÉFÉRENCE

Arrays

- les tableaux (**Arrays**) en Java, qui sont un moyen fondamental de stocker des séquences d'éléments du même type.

- Les tableaux sont très utilisés en programmation Java pour diverses raisons, notamment leur efficacité et leur facilité d'utilisation.

LES TYPES RÉFÉRENCE(ARRAYS)

Définition et Déclaration:

Un tableau en Java est un objet qui contient un nombre fixe d'éléments d'un même type. La syntaxe pour déclarer un tableau est la suivante :

```
type[] nomDuTableau;
```

Par exemple, pour déclarer un tableau d'entiers, vous écririez :

```
int[] monTableau;
monTableau = new int[10]; // Crée un tableau de 10 entiers
Ou bien :      int[] monTableau = {1, 2, 3, 4, 5};
```

LES TYPES RÉFÉRENCE(ARRAYS)

Limitations:

- La taille d'un tableau est fixe une fois qu'il est créé. Si vous avez besoin d'une structure dont la taille peut changer dynamiquement, envisagez d'utiliser une classe du framework Collections, comme **ArrayList**.

- Les tableaux stockent des éléments du même type. Si vous avez besoin de stocker des éléments de types différents, envisagez d'utiliser une collection ou créez une classe personnalisée pour encapsuler les différents types.

Collections Framework

- Le Collections Framework en Java est un ensemble de classes et d'interfaces qui permettent de stocker et de manipuler des groupes d'objets.

- C'est comme une boîte à outils pour travailler avec des ensembles d'éléments, permettant de les stocker, de les manipuler, et de les accéder de manière efficace et organisée.

LES TYPES RÉFÉRENCE (COLLECTION FRAMEWORK)

- **List**: Une interface qui étend **Collection** et représente une liste ordonnée d'éléments. Les éléments peuvent être accédés par leur indice (position dans la liste) et la liste peut contenir des doublons.

- **Set**: Une autre extension de **Collection** qui représente un ensemble d'éléments uniques, c'est-à-dire sans doublons. Il ne garantit pas l'ordre des éléments.

- **Queue**: Elle représente une collection d'éléments destinés à être traités dans un ordre spécifique, généralement mais pas exclusivement dans un ordre FIFO (First In, First Out).

- **Map**: Bien que ne faisant pas partie de l'architecture de **Collection**, **Map** est une interface qui stocke des paires clé-valeur. Chaque clé est unique et peut être utilisée pour retrouver rapidement sa valeur associée.

LES TYPES RÉFÉRENCE (COLLECTION FRAMEWORK)

```java
// Utilisation d'une ArrayList
List<String> list = new ArrayList<>();
list.add("Java");
list.add("Python");
list.add("C++");

// Utilisation d'un HashSet
Set<Integer> set = new HashSet<>();
set.add(10);
set.add(20);
set.add(10); // Ce doublon ne sera pas ajouté

// Utilisation d'une HashMap
Map<String, Integer> map = new HashMap<>();
map.put("Alice", 30);
map.put("Bob", 25);
map.put("Charlie", 35);
```

Les classes enveloppes:

- Ces classes fournissent un moyen d'utiliser les types primitifs comme des objets.
- Chaque type primitif en Java a une classe enveloppe correspondante dans le package **java.lang**.
- Ces classes sont cruciales pour travailler avec les collections, qui ne peuvent stocker que des objets.

LES TYPES RÉFÉRENCE (CLASSES ENVELOPPES)

Liste des Classes Enveloppes

- **Boolean** : enveloppe pour le type primitif **boolean**.
- **Byte** : enveloppe pour le type primitif **byte**.
- **Character** : enveloppe pour le type primitif **char**.
- **Double** : enveloppe pour le type primitif **double**.
- **Float** : enveloppe pour le type primitif **float**.
- **Integer** : enveloppe pour le type primitif **int**.
- **Long** : enveloppe pour le type primitif **long**.
- **Short** : enveloppe pour le type primitif **short**.

LES TYPES RÉFÉRENCE (COLLECTION FRAMEWORK)

```
Integer monInteger = Integer.valueOf(10); // Crée un objet Integer
int monInt = monInteger.intValue(); // Convertit l'objet Integer en un int

// Autoboxing dans une collection
List<Integer> liste = new ArrayList<>();
liste.add(15); // L'entier est automatiquement converti en objet Integer

// Unboxing
int valeur = liste.get(0); // L'objet Integer est automatiquement converti en int
```

LES OPÉRATEURS(ARITHMÉTIQUES)

Type d'Opérateur	Opérateur	Description	Exemple
Arithmétique	+	Addition de deux valeurs.	int a = 5 + 3; // a vaut 8
	-	Soustraction d'une valeur par une autre.	int b = a - 2; // b vaut 6
	*	Multiplication de deux valeurs.	int c = b * 2; // c vaut 12
	/	Division d'une valeur par une autre.	int d = c / 3; // d vaut 4
	%	Modulo - Reste de la division d'une valeur par une autre.	int e = c % 5; // e vaut 2

LES OPÉRATEURS (UNAIRES)

Type d'Opérateur	Opérateur	Description	Exemple
Unaire	-	Négation d'une valeur.	int f = -d; // f vaut -4
	+	Indique une valeur positive (rarement utilisé).	int g = +f; // g vaut 4
	++	Incrémentation - Augmente la valeur de 1.	d++; // d vaut maintenant 5
	--	Décrémentation - Diminue la valeur de 1.	d--; // d vaut à nouveau 4
	!	Négation logique - Inverse une valeur booléenne.	boolean h = !true; // h vaut false

LES OPÉRATEURS (UNAIRES)

Type d'Opérateur	Opérateur	Description	Exemple
Affectation	=	Affecte la valeur de droite à la variable de gauche.	int i = 5; // i vaut 5
	+=	Ajoute la valeur de droite à la variable de gauche.	i += 3; // i vaut maintenant 8
	-=	Soustrait la valeur de droite de la variable de gauche.	i -= 2; // i vaut maintenant 6
	*=	Multiplie la variable de gauche par la valeur de droite.	i *= 2; // i vaut maintenant 12
	/=	Divise la variable de gauche par la valeur de droite.	i /= 4; // i vaut maintenant 3

LES OPÉRATEURS (COMPARAISON)

Type d'Opérateur	Opérateur	Description	Exemple
Comparaison	==	Teste l'égalité de deux valeurs.	boolean j = (i == 3); // j vaut true
	!=	Teste l'inégalité de deux valeurs.	boolean k = (i != 3); // k vaut false
	<	Vérifie si la valeur de gauche est inférieure à celle de droite.	boolean l = (i < 5); // l vaut true
	>	Vérifie si la valeur de gauche est supérieure à celle de droite.	boolean m = (i > 5); // m vaut false
	<=	Vérifie si la valeur de gauche est inférieure ou égale à celle de droite.	boolean n = (i <= 3); // n vaut true
	>=	Vérifie si la valeur de gauche est supérieure ou égale à celle de droite.	boolean o = (i >= 3); // o vaut true

LES OPÉRATEURS (LOGIQUE)

Type d'Opérateur	Opérateur	Description	Exemple
Logique	&&	Opérateur ET logique. Retourne true si les deux opérandes sont true.	boolean p = (j && o); // p vaut true
	||	Opérateur OU logique. Retourne true si au moins un des opérandes est true.	boolean q = (l || m); // q vaut true
	^	Opérateur OU exclusif logique. Retourne true si les opérandes sont différents.	boolean r = (j ^ k); // r vaut true

LES C

C'est e et
retou sse.

•cond

```
int a = 10, b = 5;
int result;

// Opérateur arithmétique
result = a + b; // 15

// Opérateur de comparaison
boolean isGreater = a > b; // true

// Opérateur logique
boolean andResult = (a > b) && (b > 0); // true

// Opérateur ternaire
int max = (a > b) ? a : b; // 10
```

PRIORITÉ DES OPÉRATEURS

Priorité	Opérateurs	Description
1	()	Parenthèses (groupement)
2	++, -- (préfixe/postfixe)	Opérateurs d'incrémentation et de décrémentation
3	*, /, %	Multiplication, division, modulo
4	+, -	Addition, soustraction
5	<<, >>	Opérateurs de décalage
6	<, >, <=, >=	Opérateurs de comparaison
7	==, !=	Opérateurs d'égalité
8	&	Opérateur ET (bit à bit)
9	^	Opérateur OU exclusif (bit à bit)
10	|	Opérateur OU (bit à bit)
11	&&	Opérateur ET logique
12	||	Opérateur OU logique
13	=, +=, -=, *=, /=, %=, <<=, >>=, &=, ^=, |=	Opérateurs d'assignation (affectation)

LES STRUCTURE DE CONTRÔLE

- Les structures conditionnelles sont un ensemble d'instructions qui permettent de vérifier si une condition est

vraie ou non.

- Le langage de programmation Java fournit les structures conditionnelles suivantes :
  - ***if***
  - ***if-else***
  - ***switch***

L'INSTRUCTION "IF"

- Ce type de structure est la plus basique. Elle permet d'exécuter une série ou un bloc d'instructions si la condition est vraie .
- La syntaxe de l'instruction if est la suivante:

¶ *Syntaxe*

```
1 if( condition ){
2 //
3 // corps
4 //instructions à exécuter si la condition est true
5 //
6 }
7
```

STRUCTURE CONDITIONNELLE IF-ELSE

- Dans le cas où la condition de l'instruction basique IF n'est pas réalisée, il est possible d'exécuter une autre série d'instructions grâce à l'instruction *"ELSE".*
- *La syntaxe de l'instruction if..else est la suivante:*

¶ *Syntaxe*

```
1 f( condition ){
2     //
3     // corps
4     //instructions à exécuter si la condition est true
5     //
6 }
7 else{
8
9      //
10    //corps
11    // instructions à exécuter si condition est false
12 }
```

STRUCTURE CONDITIONNELLE IF-ELSE

Exemple

```
1 public class Test{
2
3     public static void main(String args[]){
4         a=4;
5         // structure conditionnelle
6         if(a< 0){
7             System.out.println("la valeur de a est négative ");
8         }
9         else{
10            System.out.println("la valeur de a est positive ou nulle ");
11        }
12
13        // fin du programme
14        System.out.println("merci !");
15    }
16 }
```

Simulation

la valeur de a est positive ou nulle

merci !

STRUCTURE CONDITIONNELLE SWITCH

- L'instruction switch est une instruction multidirectionnelle utilisée pour gérer les décisions. Son fonctionnement est similaire à la structure conditionnelle IF-ELSE. La différence est que l'instruction switch génère un code plus lisible.
- De plus, il s'exécute parfois plus rapidement que son homologue if-else.
- Le syntaxe de la structure SWITCH est de la forme :

STRUCTURE CONDITIONNELLE SWITCH

¶ *Syntaxe*

```
1 switch(expression)
2 {
3
4     case val1:
5         // instructions
6         break; // optionnelle
7
8     case val2:
9         // instructions
10        break; // optionnelle
11
12    case val3:
13        // instructions
14        break; // optionnelle
15
16    default:
17        // instructions
18 }
```

STRUCTURE CONDITIONNELLE SWITCH

Règles:

- ✔ Les variables utilisées dans une instruction switch (ou expression) doivent être nombres *entiers*, des nombres *entiers convertibles (byte, short, char*), des *chaînes* et des *enums*.
- ✔ La valeur d'une case doit être du même type de données que la variable(ou expression) dans le switch.
- ✔ Lorsque la variable est égale à un cas, les instructions qui suivent ce cas seront exécutées jusqu'à ce qu'une instruction break soit atteinte.
- ✔ Lorsqu'une instruction break est atteinte, switch se termine et le flux de contrôle passe à la ligne suivante qui suit l'instruction switch.

LES STRUCTURES RÉPÉTITIVES

En programmation, une boucle permet de répéter l'exécution d'un bloc jusqu'à ce qu'une certaine condition soit atteinte.

Il existe principalement deux types de boucles:

- ***Boucles à entrée contrôlée:*** dans ce type de boucles, la condition est testée avant d'entrer dans le corps de la boucle. boucle *for* et boucle *while* sont des boucles d'entrée contrôlées.

- ***Boucles à sortie contrôlée:*** dans ce type de boucles, la condition est testée ou évaluée à l'extrémité du corps de la boucle. Par conséquent, le corps de boucle s'exécutera au moins une fois, que la condition soit true ou false. *do-while* est une boucle de sortie contrôlé.

BOUCLE FOR

¶ *Syntaxe*

```
1 for(initialisation ; condition ; mise à jour){
2
3     // bloc d'instructions à répéter
4
5 }
```

BOUCLE WHILE…DO

¶ *Syntaxe*

```
1 initialisation
2 while(condition){
3
4
5     // corps de la bucle
6
7
8     mise à jour
9 }
```

BOUCLE DO…WHILE

¶ *Syntaxe*

```
1 initialisation
2 do{
3     // corps de la bucle
4
5     mise à jour
6 }while(condition);
```

CONSEIL

- la boucle *for* est utile lorsque le *nombre d'itérations est connu* à l'avance.
- les boucles *while* pour lesquelles *le nombre exact d'itérations n'est pas connu* mais la *condition de fin de boucle est connue*.
- Utilisez la boucle *do-while* si le *code doit être exécuté au moins une fois*.